



Introduction

- Pandas is an open- source python library for data analysis and manipulation. It provides data structures like Series and DataFrame for handling structured data .

Installation and Import

```
!pip install pandas
```

```
import pandas as pd
```

Series

- A one dimensional labeled array capable of holding any data type (integer,float,string etc..)

```
import pandas as pd
import numpy as np

p = pd.Series ([1,2,3,4,"ganesh",23.90])

p
0      1
1      2
2      3
3      4
4  ganesh
5    23.9
dtype: object

names = ["ganesh","sai","vamsi","venky","raju"]
names = pd.Series(names,index = [101,102,103,104,105])
names

101    ganesh
102      sai
103    vamsi
104    venky
105    raju
dtype: object
```

DataFrame

- A two dimensional labeled data structure with columns of potentially different types .

```
## create data frame  ##column wise
## from dictionary
ipl = {"player" :["virat","axar","pant","siraj"],
       "team":["rcb","csk","csk","csk"],
       "age": [34,54,32,43]}

pd.DataFrame(ipl)
```

```

    player team  age
0  virat  rcb    34
1  axar   csk    54
2  pant   csk    32
3  siraj   csk    43

##rowwise dataframe
fruits = [
    ["apple", "red", 100],
    ["mango", "yellow", 30],
    ["grapes", "black", 90],
    ["dragon", "pink", 80]
]
pd.DataFrame(fruits, columns = ["name", "colors", "price"])

```

	name	colors	price
0	apple	red	100
1	mango	yellow	30
2	grapes	black	90
3	dragon	pink	80

DataInput / DataOutput

CSV file

- Reading :
- `df=pd.read_csv(r"filename.csv")`

Excel

- Reading
- `df = pd.read_excel(r"filename.xlsx")`

Json

- Reading
- `df = pd.read_json(r"filename.json")`

Example of opening csv file

```

df=pd.read_csv(r"C:\Users\91798\Desktop\loandata.csv")
df

```

	Loan_ID	Gender	Married	Dependents	Education	
Self_Employed						
0	LP001722	Male	Yes	0	Graduate	No
1	LP002502	Female	Yes	2	Not Graduate	NaN
2	LP002949	Female	No	3+	Graduate	NaN
3	LP002603	Female	No	0	Graduate	No
4	LP001644	NaN	Yes	0	Graduate	Yes

..	...	...	...	...	...	...
609	LP001640	Male	Yes	0	Graduate	Yes
610	LP001536	Male	Yes	3+	Graduate	No
611	LP001585	NaN	Yes	3+	Graduate	No
612	LP002101	Male	Yes	0	Graduate	NaN
613	LP002317	Male	Yes	3+	Graduate	No
	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term		
\						
0	150	1800.0	135.0	360.0		
1	210	2917.0	98.0	360.0		
2	416	41667.0	350.0	180.0		
3	645	3683.0	113.0	480.0		
4	674	5296.0	168.0	360.0		
..	...	...	...	...		
609	39147	4750.0	120.0	360.0		
610	39999	0.0	600.0	180.0		
611	51763	0.0	700.0	300.0		
612	63337	0.0	490.0	180.0		
613	81000	0.0	360.0	360.0		
	Credit_History	Property_Area	Loan_Status			
0	1.0	Rural	N			
1	1.0	Semiurban	Y			
2	NaN	Urban	N			
3	1.0	Rural	Y			
4	1.0	Rural	Y			
..	...	...	...			
609	1.0	Semiurban	Y			
610	0.0	Semiurban	Y			
611	1.0	Urban	Y			
612	1.0	Urban	Y			
613	0.0	Rural	N			

[614 rows x 13 columns]

Data exploration

- view first five rows.

```
df.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001722	Male	Yes	0	Graduate	No	
1	LP002502	Female	Yes	2	Not Graduate	NaN	
2	LP002949	Female	No	3+	Graduate	NaN	
3	LP002603	Female	No	0	Graduate	No	
4	LP001644	NaN	Yes	0	Graduate	Yes	
	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\		
0	150	1800.0	135.0	360.0			
1	210	2917.0	98.0	360.0			
2	416	41667.0	350.0	180.0			
3	645	3683.0	113.0	480.0			
4	674	5296.0	168.0	360.0			
	Credit_History	Property_Area	Loan_Status				
0	1.0	Rural	N				
1	1.0	Semiurban	Y				
2	NaN	Urban	N				
3	1.0	Rural	Y				
4	1.0	Rural	Y				

- view last (bottom) fiverows

```
df.tail()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
609	LP001640	Male	Yes	0	Graduate	Yes	
610	LP001536	Male	Yes	3+	Graduate	No	
611	LP001585	NaN	Yes	3+	Graduate	No	
612	LP002101	Male	Yes	0	Graduate	NaN	
613	LP002317	Male	Yes	3+	Graduate	No	
	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\		
609	39147	4750.0	120.0	360.0			
610	39999	0.0	600.0	180.0			
611	51763	0.0	700.0	300.0			
612	63337	0.0	490.0	180.0			

```
613          81000          0.0      360.0      360.0
```

```
      Credit_History Property_Area Loan_Status
609             1.0     Semiurban           Y
610             0.0     Semiurban           Y
611             1.0           Urban           Y
612             1.0           Urban           Y
613             0.0           Rural           N
```

```
df.shape
```

```
(614, 13)
```

Columns Names

```
df.columns
```

```
Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
       'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome',
       'LoanAmount',
       'Loan_Amount_Term', 'Credit_History', 'Property_Area',
       'Loan_Status'],
      dtype='object')
```

Dataset information

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Loan_ID                614 non-null   object 
1   Gender                 601 non-null   object 
2   Married                611 non-null   object 
3   Dependents             599 non-null   object 
4   Education              614 non-null   object 
5   Self_Employed          582 non-null   object 
6   ApplicantIncome        614 non-null   int64  
7   CoapplicantIncome      614 non-null   float64 
8   LoanAmount             592 non-null   float64 
9   Loan_Amount_Term       600 non-null   float64 
10  Credit_History         564 non-null   float64 
11  Property_Area          614 non-null   object 
12  Loan_Status            614 non-null   object 
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
```

Summary statistics

```
df.describe()
```

	ApplicantIncome	CoapplicantIncome	LoanAmount
Loan_Amount_Term \			
count	614.000000	614.000000	592.000000
600.000000			
mean	5403.459283	1621.245798	146.412162
342.000000			
std	6109.041673	2926.248369	85.587325
65.12041			
min	150.000000	0.000000	9.000000
12.000000			
25%	2877.500000	0.000000	100.000000
360.000000			
50%	3812.500000	1188.500000	128.000000
360.000000			
75%	5795.000000	2297.250000	168.000000
360.000000			
max	81000.000000	41667.000000	700.000000
480.000000			

	Credit_History
count	564.000000
mean	0.842199
std	0.364878
min	0.000000
25%	1.000000
50%	1.000000
75%	1.000000
max	1.000000

```
df.describe(include = 'all',percentiles = [0,0.5])
```

It describes the both categorical and numerical data of the dataset.
It gives the statistical measures of the data.

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
count	614	601	611	599	614	582	
unique	614	2	2	4	2	2	
top	LP001722	Male	Yes	0	Graduate	No	
freq	1	489	398	345	480	500	
mean	NaN	NaN	NaN	NaN	NaN	NaN	
std	NaN	NaN	NaN	NaN	NaN	NaN	
min	NaN	NaN	NaN	NaN	NaN	NaN	
0%	NaN	NaN	NaN	NaN	NaN	NaN	
50%	NaN	NaN	NaN	NaN	NaN	NaN	
max	NaN	NaN	NaN	NaN	NaN	NaN	

	ApplicantIncome	CoapplicantIncome	LoanAmount
Loan_Amount_Term \			

count	614.000000	614.000000	592.000000
600.000000			
unique	NaN	NaN	NaN
NaN			
top	NaN	NaN	NaN
NaN			
freq	NaN	NaN	NaN
NaN			
mean	5403.459283	1621.245798	146.412162
342.000000			
std	6109.041673	2926.248369	85.587325
65.12041			
min	150.000000	0.000000	9.000000
12.000000			
0%	150.000000	0.000000	9.000000
12.000000			
50%	3812.500000	1188.500000	128.000000
360.000000			
max	81000.000000	41667.000000	700.000000
480.000000			

	Credit_History	Property_Area	Loan_Status
count	564.000000	614	614
unique	NaN	3	2
top	NaN	Semiurban	Y
freq	NaN	233	422
mean	0.842199	NaN	NaN
std	0.364878	NaN	NaN
min	0.000000	NaN	NaN
0%	0.000000	NaN	NaN
50%	1.000000	NaN	NaN
max	1.000000	NaN	NaN

Datatype

df.dtypes

Loan_ID	object
Gender	object
Married	object
Dependents	object
Education	object
Self_Employed	object
ApplicantIncome	int64
CoapplicantIncome	float64
LoanAmount	float64
Loan_Amount_Term	float64
Credit_History	float64
Property_Area	object


```
Loan_Status      object
dtype: object
```

Accessing columns

```
df.Gender
```

```
0      Male
1     Female
2     Female
3     Female
4        NaN
...
609     Male
610     Male
611        NaN
612     Male
613     Male
Name: Gender, Length: 614, dtype: object
```

```
df.Dependents
```

```
0      0
1      2
2     3+
3      0
4      0
...
609     0
610    3+
611    3+
612     0
613    3+
Name: Dependents, Length: 614, dtype: object
```

```
type(df.CoapplicantIncome)
```

```
pandas.core.series.Series
```

```
df["ApplicantIncome"]    ##AS series
```

```
0      150
1     210
2     416
3     645
4     674
...
609   39147
610   39999
611   51763
612   63337
```

```
613      81000
Name: ApplicantIncome, Length: 614, dtype: int64
```

```
df[["ApplicantIncome"]] ##as dataframe
```

	ApplicantIncome
0	150
1	210
2	416
3	645
4	674
...	...
609	39147
610	39999
611	51763
612	63337
613	81000

```
[614 rows x 1 columns]
```

```
df[["ApplicantIncome","LoanAmount","Gender"]] ###we can access multiple columns
```

	ApplicantIncome	LoanAmount	Gender
0	150	135.0	Male
1	210	98.0	Female
2	416	350.0	Female
3	645	113.0	Female
4	674	168.0	NaN
...	...	...	...
609	39147	120.0	Male
610	39999	600.0	Male
611	51763	700.0	NaN
612	63337	490.0	Male
613	81000	360.0	Male

```
[614 rows x 3 columns]
```

```
df.at[10,"Gender"] ## access a single value for a row /column label pair
```

```
'Male'
```

```
df.iat[2,3] ##index base
```

```
'3+'
```

```
## we can assign (update)the data also
```

```
df.iat[2,3] = 4
```

```
df.iat[2,3]
```

Check for missing values

```
df.isna().sum()
```

```
Loan_ID      0
Gender       13
Married       3
Dependents   15
Education     0
Self_Employed 32
ApplicantIncome 0
CoapplicantIncome 0
LoanAmount   22
Loan_Amount_Term 14
Credit_History 50
Property_Area 0
Loan_Status  0
dtype: int64
```

select rows by index

```
df.iloc[4,2]    ### indexing wise
```

```
'Yes'
```

```
df.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001722	Male	Yes	0	Graduate	No	
1	LP002502	Female	Yes	2	Not Graduate	NaN	
2	LP002949	Female	No	4	Graduate	NaN	
3	LP002603	Female	No	0	Graduate	No	
4	LP001644	NaN	Yes	0	Graduate	Yes	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	150	1800.0	135.0	360.0	
1	210	2917.0	98.0	360.0	
2	416	41667.0	350.0	180.0	
3	645	3683.0	113.0	480.0	
4	674	5296.0	168.0	360.0	

	Credit_History	Property_Area	Loan_Status
0	1.0	Rural	N
1	1.0	Semiurban	Y
2	NaN	Urban	N
3	1.0	Rural	Y
4	1.0	Rural	Y

```
df.loc[3,"Gender"]    ## name the labeling to slice
```

```
'Female'
```

unique values

```
df["Gender"].unique()  
array(['Male', 'Female', nan], dtype=object)
```

Statistical measures

```
df["ApplicantIncome"].mean()  
5403.459283387622  
df["ApplicantIncome"].median()  
3812.5  
df["Gender"].mode() ## most repeated value  
0    Male  
Name: Gender, dtype: object  
df["ApplicantIncome"].mean()  
5403.459283387622  
df["ApplicantIncome"].skew()  
6.539513113994621  
df["ApplicantIncome"].max()  
81000  
df["ApplicantIncome"].min()  
150  
df["ApplicantIncome"].quantile(0.25)  
2877.5  
df["ApplicantIncome"].quantile(0.5)  
3812.5  
df["ApplicantIncome"].quantile(0.75)  
5795.0  
df["ApplicantIncome"].var()  
37320390.16718121
```

```
df["ApplicantIncome"].std()
```

```
6109.041673387178
```

```
df.describe()
```

	ApplicantIncome	CoapplicantIncome	LoanAmount
Loan_Amount_Term \			
count	614.000000	614.000000	592.000000
600.000000			
mean	5403.459283	1621.245798	146.412162
342.000000			
std	6109.041673	2926.248369	85.587325
65.12041			
min	150.000000	0.000000	9.000000
12.000000			
25%	2877.500000	0.000000	100.000000
360.000000			
50%	3812.500000	1188.500000	128.000000
360.000000			
75%	5795.000000	2297.250000	168.000000
360.000000			
max	81000.000000	41667.000000	700.000000
480.000000			

	Credit_History
count	564.000000
mean	0.842199
std	0.364878
min	0.000000
25%	1.000000
50%	1.000000
75%	1.000000
max	1.000000

Data Manipulation

Add a new column

```
df["Genderwise _education"] = df["Gender"] + df["Education"]
```

```
df.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001722	Male	Yes	0	Graduate	No	
1	LP002502	Female	Yes	2	Not Graduate	NaN	
2	LP002949	Female	No	4	Graduate	NaN	
3	LP002603	Female	No	0	Graduate	No	
4	LP001644	NaN	Yes	0	Graduate	Yes	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
--	-----------------	-------------------	------------	------------------	---

0	150	1800.0	135.0	360.0
1	210	2917.0	98.0	360.0
2	416	41667.0	350.0	180.0
3	645	3683.0	113.0	480.0
4	674	5296.0	168.0	360.0
Credit_History Property_Area Loan_Status Genderwise_education				
0	1.0	Rural	N	MaleGraduate
1	1.0	Semiurban	Y	FemaleNot Graduate
2	NaN	Urban	N	FemaleGraduate
3	1.0	Rural	Y	FemaleGraduate
4	1.0	Rural	Y	NaN

Drop column

```
df = df.drop(["Genderwise_education"],axis = 1)
```

```
df.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001722	Male	Yes	0	Graduate	No	
1	LP002502	Female	Yes	2	Not Graduate	NaN	
2	LP002949	Female	No	4	Graduate	NaN	
3	LP002603	Female	No	0	Graduate	No	
4	LP001644	NaN	Yes	0	Graduate	Yes	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	150	1800.0	135.0	360.0	
1	210	2917.0	98.0	360.0	
2	416	41667.0	350.0	180.0	
3	645	3683.0	113.0	480.0	
4	674	5296.0	168.0	360.0	

	Credit_History	Property_Area	Loan_Status
0	1.0	Rural	N
1	1.0	Semiurban	Y
2	NaN	Urban	N
3	1.0	Rural	Y
4	1.0	Rural	Y

Rename columns

```
df=df.rename(columns ={"Married" : "Married_status"})
```

```
df.head()
```

	Loan_ID	Gender	Married_status	Dependents	Education	Self_Employed	\
0	LP001722	Male	Yes	0	Graduate	No	
1	LP002502	Female	Yes	2	Not Graduate		

NaN					
2	LP002949	Female	No	4	Graduate
NaN					
3	LP002603	Female	No	0	Graduate
No					
4	LP001644	NaN	Yes	0	Graduate
Yes					

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term \
0	150	1800.0	135.0	360.0
1	210	2917.0	98.0	360.0
2	416	41667.0	350.0	180.0
3	645	3683.0	113.0	480.0
4	674	5296.0	168.0	360.0

	Credit_History	Property_Area	Loan_Status
0	1.0	Rural	N
1	1.0	Semiurban	Y
2	NaN	Urban	N
3	1.0	Rural	Y
4	1.0	Rural	Y

```
df["Married_status"].value_counts()
```

```
Married_status
```

```
Yes    398
```

```
No     213
```

```
Name: count, dtype: int64
```

```
sum((df["Gender"] == "Female")      & (df["Married_status"] == "Yes"))
```

```
31
```

```
sum((df["Gender"] == "Male")        & (df["Property_Area"] == "Rural"))
```

```
151
```

```
df.drop(index = 0 , axis = 0 ,inplace = True)  ## inplace its delete
the original data from the given set
```

```
df.head()
```

	Loan_ID	Gender	Married_status	Dependents	Education
Self_Employed \					
1	LP002502	Female	Yes	2	Not Graduate
NaN					
2	LP002949	Female	No	4	Graduate
NaN					
3	LP002603	Female	No	0	Graduate
No					
4	LP001644	NaN	Yes	0	Graduate

```

Yes
5 LP001259 Male Yes 1 Graduate
Yes

```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term \
1	210	2917.0	98.0	360.0
2	416	41667.0	350.0	180.0
3	645	3683.0	113.0	480.0
4	674	5296.0	168.0	360.0
5	1000	3022.0	110.0	360.0

	Credit_History	Property_Area	Loan_Status
1	1.0	Semiurban	Y
2	NaN	Urban	N
3	1.0	Rural	Y
4	1.0	Rural	Y
5	1.0	Urban	N

```

df.reset_index(drop = True,inplace = True) ##get original data
df.head()

```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed \
0	LP001722	Male	Yes	0	Graduate	No
1	LP002502	Female	Yes	2	Not Graduate	NaN
2	LP002949	Female	No	3+	Graduate	NaN
3	LP002603	Female	No	0	Graduate	No
4	LP001644	NaN	Yes	0	Graduate	Yes

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term \
0	150	1800.0	135.0	360.0
1	210	2917.0	98.0	360.0
2	416	41667.0	350.0	180.0
3	645	3683.0	113.0	480.0
4	674	5296.0	168.0	360.0

	Credit_History	Property_Area	Loan_Status
0	1.0	Rural	N
1	1.0	Semiurban	Y
2	NaN	Urban	N
3	1.0	Rural	Y
4	1.0	Rural	Y

pd.Concat

```

df = pd.DataFrame({"names" : ["ganesh","prabhas","darling"],
                    "exp": [2,3,4],
                    "sal": [10000,20000,30000]})

df1 = pd.DataFrame({"GEnderr" : ["male","female","male"],
                    "mar": ["un","ma","m"]})

```



```
df2 = pd.concat((df,df1),axis = 1)
```

```
df2
```

	names	exp	sal	GENDER	mar
0	ganesh	2	10000	male	un
1	prabhas	3	20000	female	ma
2	darling	4	30000	male	m

Merging the Data

```
df1 = pd.DataFrame({"Name":["ganesh","Sai","venky","ramu"],  
                    "Company":["IBM","Google","IBM","Infosys"],  
                    "Country":["India","India","USA","UK"] })
```

```
df1
```

	Name	Company	Country
0	ganesh	IBM	India
1	Sai	Google	India
2	venky	IBM	USA
3	ramu	Infosys	UK

```
df2 = pd.DataFrame({'Name': ['sai','megha','sandy'],  
                    'Company': ['Accenture','IBM','Google'],  
                    'Country': ['India','India','India'],  
                    'Salary' : [45000,50000,60000]})
```

```
df2
```

	Name	Company	Country	Salary
0	sai	Accenture	India	45000
1	megha	IBM	India	50000
2	sandy	Google	India	60000

```
pd.merge(df1,df2, on = "Company" ,how="inner")
```

	Name_x	Company	Country_x	Name_y	Country_y	Salary
0	ganesh	IBM	India	megha	India	50000
1	Sai	Google	India	sandy	India	60000
2	venky	IBM	USA	megha	India	50000

```
pd.merge(df1,df2, on = "Country" ,how="inner")
```

	Name_x	Company_x	Country	Name_y	Company_y	Salary
0	ganesh	IBM	India	sai	Accenture	45000
1	ganesh	IBM	India	megha	IBM	50000
2	ganesh	IBM	India	sandy	Google	60000
3	Sai	Google	India	sai	Accenture	45000
4	Sai	Google	India	megha	IBM	50000
5	Sai	Google	India	sandy	Google	60000

```
pd.merge(df1,df2, on = "Country" ,how="outer")
```

	Name_x	Company_x	Country	Name_y	Company_y	Salary
0	ganesh	IBM	India	sai	Accenture	45000.0
1	ganesh	IBM	India	megha	IBM	50000.0
2	ganesh	IBM	India	sandy	Google	60000.0
3	Sai	Google	India	sai	Accenture	45000.0
4	Sai	Google	India	megha	IBM	50000.0
5	Sai	Google	India	sandy	Google	60000.0
6	ramu	Infosys	UK	NaN	NaN	NaN
7	venky	IBM	USA	NaN	NaN	NaN

```
pd.merge(df1,df2, on = "Country" ,how="right")
```

	Name_x	Company_x	Country	Name_y	Company_y	Salary
0	ganesh	IBM	India	sai	Accenture	45000
1	Sai	Google	India	sai	Accenture	45000
2	ganesh	IBM	India	megha	IBM	50000
3	Sai	Google	India	megha	IBM	50000
4	ganesh	IBM	India	sandy	Google	60000
5	Sai	Google	India	sandy	Google	60000

```
pd.merge(df1,df2, on = "Country" ,how="left")
```

	Name_x	Company_x	Country	Name_y	Company_y	Salary
0	ganesh	IBM	India	sai	Accenture	45000.0
1	ganesh	IBM	India	megha	IBM	50000.0
2	ganesh	IBM	India	sandy	Google	60000.0
3	Sai	Google	India	sai	Accenture	45000.0
4	Sai	Google	India	megha	IBM	50000.0
5	Sai	Google	India	sandy	Google	60000.0
6	venky	IBM	USA	NaN	NaN	NaN
7	ramu	Infosys	UK	NaN	NaN	NaN

```
df2.sort_values(by = "Salary",ascending = False,ignore_index = True,inplace = True)  ## descending order salary highest to lowest
```

```
df2
```

	Name	Company	Country	Salary
0	sandy	Google	India	60000
1	megha	IBM	India	50000
2	sai	Accenture	India	45000

Time Series

```
df = pd.read_csv(r"C:\Users\91798\Desktop\time_series.csv")
```

```
df
```

	ID	Datetime	Count
0	0	25-08-2012 00:00	8
1	1	25-08-2012 01:00	2
2	2	25-08-2012 02:00	6
3	3	25-08-2012 03:00	2
4	4	25-08-2012 04:00	2
...	...	...	...
18283	18283	25-09-2014 19:00	868
18284	18284	25-09-2014 20:00	732
18285	18285	25-09-2014 21:00	702
18286	18286	25-09-2014 22:00	580
18287	18287	25-09-2014 23:00	534

[18288 rows x 3 columns]

df["Datetime"]

0	25-08-2012 00:00
1	25-08-2012 01:00
2	25-08-2012 02:00
3	25-08-2012 03:00
4	25-08-2012 04:00

18283	25-09-2014 19:00
18284	25-09-2014 20:00
18285	25-09-2014 21:00
18286	25-09-2014 22:00
18287	25-09-2014 23:00

Name: Datetime, Length: 18288, dtype: object

import warnings

warnings.filterwarnings("ignore")

df2

	Name	Company	Country	Salary
0	sandy	Google	India	60000
1	megha	IBM	India	50000
2	sai	Accenture	India	45000

df2["Salary"].nsmallest(1)

2 45000

Name: Salary, dtype: int64

df2["Salary"].nlargest(1)

0 60000

Name: Salary, dtype: int64

Group by

```
df=pd.read_csv(r"C:\Users\91798\Desktop\loandata.csv")
```

```
df.head()
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001722	Male	Yes	0	Graduate	No	
1	LP002502	Female	Yes	2	Not Graduate	NaN	
2	LP002949	Female	No	3+	Graduate	NaN	
3	LP002603	Female	No	0	Graduate	No	
4	LP001644	NaN	Yes	0	Graduate	Yes	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	150	1800.0	135.0	360.0	
1	210	2917.0	98.0	360.0	
2	416	41667.0	350.0	180.0	
3	645	3683.0	113.0	480.0	
4	674	5296.0	168.0	360.0	

	Credit_History	Property_Area	Loan_Status
0	1.0	Rural	N
1	1.0	Semiurban	Y
2	NaN	Urban	N
3	1.0	Rural	Y
4	1.0	Rural	Y

```
df.groupby(by = "Gender")["ApplicantIncome"].agg([np.mean,np.max])
```

	mean	max
Gender		
Female	4643.473214	19484
Male	5446.460123	81000

```
df.groupby(by = "Gender")  
[["ApplicantIncome","LoanAmount"]].agg([np.mean,np.max])
```

	ApplicantIncome		LoanAmount	
	mean	max	mean	max
Gender				
Female	4643.473214	19484	126.697248	600.0
Male	5446.460123	81000	149.265957	650.0

```
## getting the average income of loanapplicant from different areas
```

```
df.groupby(by = "Property_Area")["ApplicantIncome"].agg(np.mean)
```

```
Property_Area  
Rural      5554.083799  
Semiurban  5292.261803  
Urban      5398.247525  
Name: ApplicantIncome, dtype: float64
```

Pivot Table

```
pd.pivot_table(data = df, values = "ApplicantIncome", index = "Property_Area", aggfunc=["mean", "std"])
```

Property_Area	mean	std
	ApplicantIncome	ApplicantIncome
Rural	5554.083799	6782.658637
Semiurban	5292.261803	5279.629359
Urban	5398.247525	6392.928779

```
pd.pivot_table(data = df, values = ["ApplicantIncome", "LoanAmount"], index = ["Gender", "Married"], aggfunc=["mean", "std"])
```

		mean			std
		ApplicantIncome	LoanAmount	ApplicantIncome	LoanAmount
Female	No	4503.787500	116.115385	3333.160790	57.781042
	Yes	4829.645161	153.322581	4152.831039	114.183883
Male	No	5236.146154	136.088000	4379.170994	81.771516
	Yes	5529.540616	154.011662	6743.209021	83.025254

MultiIndexing

```
outer = ["A", "A", "A", "B", "B", "B "]
inner = [1, 2, 3, 1, 2, 3]
list(zip(outer, inner))
```

```
[('A', 1), ('A', 2), ('A', 3), ('B', 1), ('B', 2), ('B ', 3)]
```

```
multi= list(zip(outer, inner))
```

```
index = pd.MultiIndex.from_tuples(multi)
```

```
a=np.random.randint(1,100,6)
```

```
df = pd.DataFrame(a, index = index)
```

```
df
```

```
      0
A  1  87
   2  12
   3  48
B  1  48
   2  14
B  3  59
```

```
df.loc["A"]
```

```

0
1 87
2 12
3 48

df.loc["B"].loc[2]

0    14
Name: 2, dtype: int32

```

Data Cleaning

Identify Missing values

```

df = pd.read_csv(r"C:\Users\91798\Desktop\SampleWeatherData.csv")

df

```

	Time	Temperature	Humidity
0	2023-04-01	22.0	55.0
1	2023-04-02	NaN	NaN
2	2023-04-03	NaN	60.0
3	2023-04-04	25.0	NaN
4	2023-04-05	NaN	65.0
5	2023-04-06	28.0	NaN
6	2023-04-07	29.0	70.0
7	2023-04-08	NaN	NaN
8	2023-04-09	NaN	75.0
9	2023-04-10	30.0	NaN

```
df.isna().sum()
```

```

Time      0
Temperature  5
Humidity   5
dtype: int64

```

```
df.dropna()
```

	Time	Temperature	Humidity
0	2023-04-01	22.0	55.0
6	2023-04-07	29.0	70.0

```
df.dropna(axis= 1)
```

	Time
0	2023-04-01
1	2023-04-02
2	2023-04-03
3	2023-04-04
4	2023-04-05

```
5 2023-04-06
6 2023-04-07
7 2023-04-08
8 2023-04-09
9 2023-04-10
```

```
df
```

	Time	Temperature	Humidity
0	2023-04-01	22.0	55.0
1	2023-04-02	NaN	NaN
2	2023-04-03	NaN	60.0
3	2023-04-04	25.0	NaN
4	2023-04-05	NaN	65.0
5	2023-04-06	28.0	NaN
6	2023-04-07	29.0	70.0
7	2023-04-08	NaN	NaN
8	2023-04-09	NaN	75.0
9	2023-04-10	30.0	NaN

```
df["Temperature"].mean()
```

```
26.8
```

```
df["Temperature"].fillna(df["Temperature"].mean(),inplace = True)
##using fill na we call the nan values
```

```
df
```

	Time	Temperature	Humidity
0	2023-04-01	22.0	55.0
1	2023-04-02	26.8	NaN
2	2023-04-03	26.8	60.0
3	2023-04-04	25.0	NaN
4	2023-04-05	26.8	65.0
5	2023-04-06	28.0	NaN
6	2023-04-07	29.0	70.0
7	2023-04-08	26.8	NaN
8	2023-04-09	26.8	75.0
9	2023-04-10	30.0	NaN

```
df["Humidity"].median()
```

```
65.0
```

```
import warnings
warnings.filterwarnings('ignore')

df["Humidity"].fillna(df["Humidity"].median(),inplace = True)

df
```

	Time	Temperature	Humidity
0	2023-04-01	22.0	55.0
1	2023-04-02	26.8	65.0
2	2023-04-03	26.8	60.0
3	2023-04-04	25.0	65.0
4	2023-04-05	26.8	65.0
5	2023-04-06	28.0	65.0
6	2023-04-07	29.0	70.0
7	2023-04-08	26.8	65.0
8	2023-04-09	26.8	75.0
9	2023-04-10	30.0	65.0

```
df.isna().sum()
```

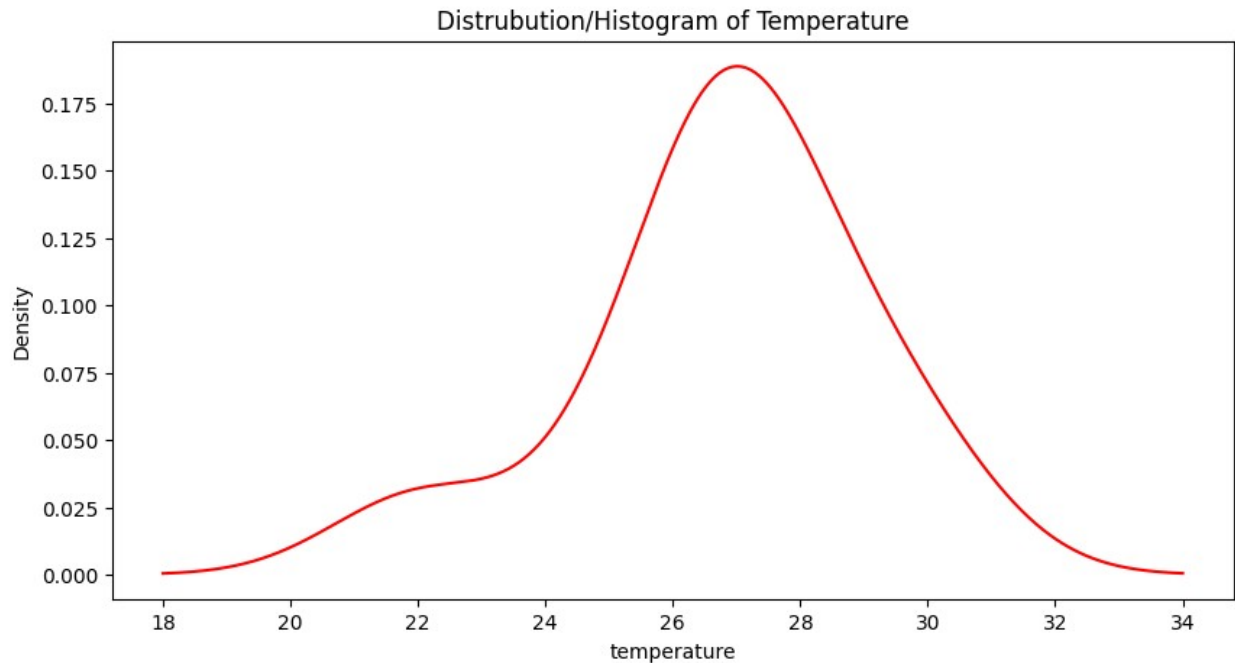
```
Time          0
Temperature    0
Humidity       0
dtype: int64
```

Data Visualization

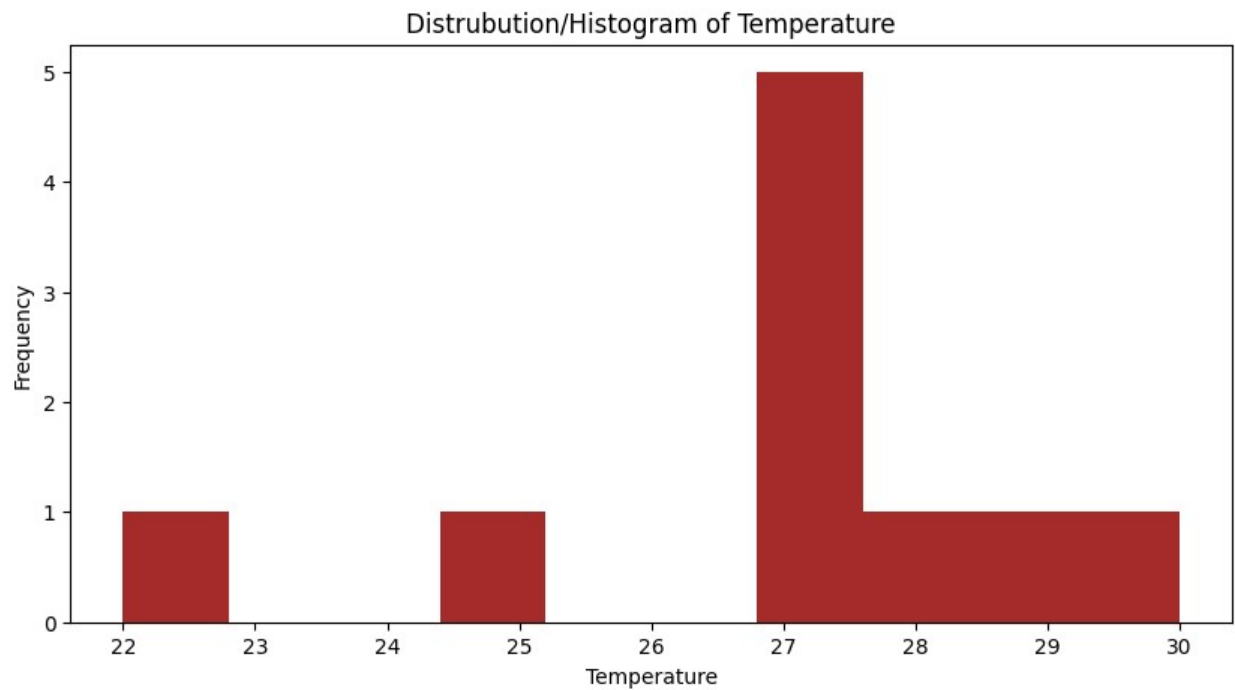
```
## ! pip install seaborn
import seaborn as sns
## !pip install matplotlib
import matplotlib.pyplot as plt
```

UNIVARIATE ANALYSIS

```
plt.figure(figsize = (10,5))  ##to change the plot
df["Temperature"].plot(kind = "kde",color = "red")
plt.xlabel("temperature")  #3 labeling the x axis name
plt.title("Distrubution/Histogram of Temperature")  ## labeling the
title of the plot
plt.show()  ## displaying the plot
```

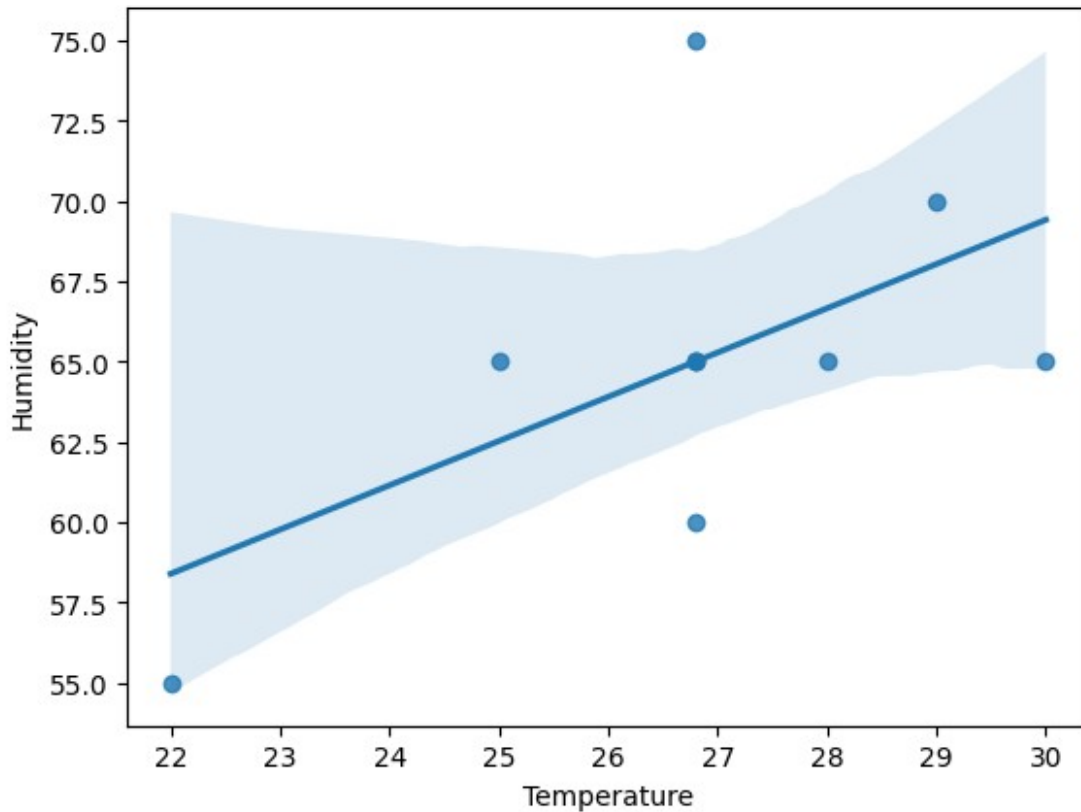
```
plt.figure(figsize = (10,5)) ##to change the plot
df["Temperature"].plot(kind = "hist",color = "brown")
plt.xlabel("Temperature") #3 labeling the x axis name
plt.title("Distrubution/Histogram of Temperature") ## labeling the
title of the plot
plt.show() ## displaying the plot
```



using Seaborn

##Bivariate Analysis

```
sns.regplot(data = df,x ="Temperature",y = "Humidity")  
<Axes: xlabel='Temperature', ylabel='Humidity'>
```



Multivariate Analysis

```
plt.figure(figsize = (5,6))  
sns.heatmap(df.corr(numeric_only = True),annot = True)  
<Axes: >
```

